Manipulación de DataFrames con Pandas

Edison Achalma
Escuela Profesional de Economía, Universidad Nacional de San Cristóbal de Huamanga

Resumen

Este abstract será actualizado una vez que se complete el contenido final del artículo.

Tabla de contenidos

Introduction **3**

1 DataFrames **3**

2 Creación de DataFrames **3**
2.1 Desde diccionarios . . . 3
2.2 Desde listas de listas . . . 4
2.3 Desde Series . . . 5
2.4 Con índice personalizado . . . 5
2.5 DataFrames de ejemplo . . . 5

3 Importación y exportación de datos **6**
3.1 Leer archivos CSV . . . 6
3.2 Leer archivos Excel . . . 7
3.3 Exportar datos . . . 7
3.4 Leer desde otras fuentes . . . 8

4 Selección y filtrado de datos **8**
4.1 Información básica del DataFrame . . . 8
4.2 Seleccionar columnas . . . 10
4.3 Seleccionar filas con loc e iloc . . . 11
4.4 Filtrado por condiciones . . . 13
4.5 Query (consultas tipo SQL) . . . 14

Edison Achalma https://orcid.org/0000-0001-6996-3364

El autor no tiene conflictos de interés que revelar. Los roles de autor se clasificaron utilizando la taxonomía de roles de colaborador (CRediT; https://credit.niso.org/) de la siguiente manera: Edison Achalma: conceptualización, metodología, análisis formal, investigación, recursos, curación de datos, redacción, visualización, supervisión, administración del proyecto

La correspondencia relativa a este artículo debe dirigirse a Edison Achalma, Escuela Profesional de Economía, Universidad Nacional de San Cristóbal de Huamanga, Portal Independencia N° 57, Ayacucho, AYA 5001, Perú, Email: elmer.achalma.09@unsch.edu.pe

5 Modificación de columnas **15**
5.1 Crear nuevas columnas . . . 15
5.2 Modificar columnas existentes . . . 17
5.3 Eliminar columnas . . . 18

6 Renombrar columnas **18**
6.1 Métodos de renombrado . . . 18
6.2 Aplicaciones económicas . . . 20

7 Operaciones con DataFrames **21**
7.1 Operaciones aritméticas . . . 21
7.2 Aplicar funciones . . . 22
7.3 Operaciones de agregación . . . 23

8 Merge y Join **24**
8.1 Tipos de merge . . . 25
8.2 Merge con indicador . . . 26
8.3 Merge con diferentes nombres de columna . . . 27
8.4 Merge múltiple . . . 28

9 Append y Concatenación **29**
9.1 Append (agregar filas) . . . 29
9.2 Concatenación vertical y horizontal . . . 29

10 Agrupación y agregación **31**
10.1 GroupBy básico . . . 31
10.2 GroupBy con múltiples columnas . . . 32
10.3 Funciones de agregación personalizadas . . . 33

11 Aplicaciones económicas **34**
11.1 Aplicación 1: Análisis de datos COVID-19 . . . 34
11.2 Aplicación 2: Análisis de comercio internacional . . . 37

12 Ejercicios prácticos **40**
12.1 Ejercicio 1: Análisis de indicadores macroeconómicos . . . 40
12.2 Ejercicio 2: Sistema de gestión de inventarios . . . 43

13 Conclusión **48**

14 Próximos pasos **48**

15 Recursos adicionales **48**

16 Publicaciones Similares **48**

Manipulación de DataFrames con Pandas

En esta séptima guía profundizaremos en el uso de DataFrames de Pandas, la estructura de datos más importante para análisis económico en Python. Los DataFrames nos permiten trabajar con datos tabulares de manera similar a Excel o Stata, pero con mucho más poder y flexibilidad.

1 DataFrames

Un DataFrame es una estructura de datos bidimensional similar a una tabla de Excel o una base de datos SQL. Consiste en:

- Filas (observaciones)
- Columnas (variables)
- Índices (etiquetas de filas)
- Tipos de datos por columna

```
import pandas as pd
import numpy as np

# Estructura básica de un DataFrame
print("Estructura de un DataFrame:")
print("""

     Index     Column 1   Column 2   Column 3

     0         valor      valor      valor
     1         valor      valor      valor
     2         valor      valor      valor

""")
```

```
Estructura de un DataFrame:


     Index     Column 1   Column 2   Column 3

     0         valor      valor      valor
     1         valor      valor      valor
     2         valor      valor      valor
```

2 Creación de DataFrames

2.1 Desde diccionarios

```
# Crear DataFrame desde diccionario
datos = {
    'pais': ['Perú', 'Chile', 'Argentina', 'Colombia', 'Brasil'],
```

```
    'pib': [242632, 301000, 487000, 314000, 1920000],
    'poblacion': [33715471, 19493184, 45810000, 51516562, 214326223],
    'continente': ['América del Sur'] * 5
}

df_paises = pd.DataFrame(datos)
print("DataFrame de países:")
print(df_paises)

# Información del DataFrame
print(f"\nForma: {df_paises.shape}")
print(f"Columnas: {df_paises.columns.tolist()}")
print(f"\nTipos de datos:")
print(df_paises.dtypes)
```

```
DataFrame de países:
        pais      pib  poblacion       continente
0       Perú   242632   33715471  América del Sur
1      Chile   301000   19493184  América del Sur
2  Argentina   487000   45810000  América del Sur
3   Colombia   314000   51516562  América del Sur
4     Brasil  1920000  214326223  América del Sur

Forma: (5, 4)
Columnas: ['pais', 'pib', 'poblacion', 'continente']

Tipos de datos:
pais          object
pib            int64
poblacion      int64
continente    object
dtype: object
```

2.2 Desde listas de listas

```
# Crear DataFrame desde listas
datos = [
    ['Perú', 242632, 33715471],
    ['Chile', 301000, 19493184],
    ['Argentina', 487000, 45810000]
]

columnas = ['pais', 'pib', 'poblacion']

df = pd.DataFrame(datos, columns=columnas)
print("\nDataFrame desde listas:")
print(df)
```

```
DataFrame desde listas:
        pais     pib  poblacion
0       Perú  242632   33715471
1      Chile  301000   19493184
2  Argentina  487000   45810000
```

2.3 Desde Series

```python
# Crear DataFrame desde Series
paises = pd.Series(['Perú', 'Chile', 'Argentina'], name='pais')
pib = pd.Series([242632, 301000, 487000], name='pib')

df = pd.DataFrame({'pais': paises, 'pib': pib})
print("\nDataFrame desde Series:")
print(df)
```

```
DataFrame desde Series:
        pais     pib
0       Perú  242632
1      Chile  301000
2  Argentina  487000
```

2.4 Con índice personalizado

```python
# DataFrame con índice personalizado
datos = {
    'producto': ['Laptop', 'Mouse', 'Teclado', 'Monitor'],
    'precio': [2500, 25, 80, 800],
    'stock': [15, 100, 50, 25]
}

df_productos = pd.DataFrame(datos, index=['PROD001', 'PROD002', 'PROD003', 'PROD004']
print("\nDataFrame con índice personalizado:")
print(df_productos)
```

```
DataFrame con índice personalizado:
        producto  precio  stock
PROD001   Laptop    2500     15
PROD002    Mouse      25    100
PROD003  Teclado      80     50
PROD004  Monitor     800     25
```

2.5 DataFrames de ejemplo

```
# Crear DataFrame de ejemplo económico
np.random.seed(42)

n = 100
df_transacciones = pd.DataFrame({
    'fecha': pd.date_range('2023-01-01', periods=n, freq='D'),
    'monto': np.random.uniform(100, 1000, n),
    'tipo': np.random.choice(['ingreso', 'gasto'], n),
    'categoria': np.random.choice(['Ventas', 'Servicios', 'Costos', 'Gastos'], n),
    'cliente_id': np.random.randint(1, 20, n)
})

print("\nDataFrame de transacciones:")
print(df_transacciones.head(10))
print(f"\nForma: {df_transacciones.shape}")
```

```
DataFrame de transacciones:
       fecha       monto     tipo  categoria  cliente_id
0 2023-01-01  437.086107  ingreso     Ventas          11
1 2023-01-02  955.642876    gasto     Ventas          17
2 2023-01-03  758.794548  ingreso     Costos           8
3 2023-01-04  638.792636  ingreso  Servicios           4
4 2023-01-05  240.416776    gasto  Servicios           6
5 2023-01-06  240.395068  ingreso     Ventas           8
6 2023-01-07  152.275251    gasto     Gastos           3
7 2023-01-08  879.558531    gasto  Servicios          16
8 2023-01-09  641.003511    gasto     Gastos           3
9 2023-01-10  737.265320  ingreso  Servicios          18

Forma: (100, 5)
```

3 Importación y exportación de datos

3.1 Leer archivos CSV

```
# Leer CSV
# df = pd.read_csv('datos.csv')

# Leer CSV con opciones
# df = pd.read_csv('datos.csv',
#                  sep=';',              # Separador
#                  encoding='utf-8',     # Codificación
#                  decimal=',',          # Separador decimal
#                  thousands='.',        # Separador de miles
#                  na_values=['NA', ''])  # Valores que representan NA

# Ejemplo con datos de INEI o BCRP
# df_pib = pd.read_csv('pib_regional.csv',
```

```
#                       encoding='latin1',
#                       thousands=',')
```

3.2 Leer archivos Excel

```
# Leer Excel
# df = pd.read_excel('datos.xlsx')

# Leer hoja específica
# df = pd.read_excel('datos.xlsx', sheet_name='Hoja1')

# Leer múltiples hojas
# df_dict = pd.read_excel('datos.xlsx', sheet_name=None)

# Leer rango específico
# df = pd.read_excel('datos.xlsx',
#                    sheet_name='Datos',
#                    usecols='A:D',
#                    skiprows=2)

# Ejemplo con datos del INEI
# df_inei = pd.read_excel('informacion_de_distritos_2020.xlsx',
#                         sheet_name='Nivel Distrital',
#                         skiprows=0)
```

3.3 Exportar datos

```
# Crear DataFrame de ejemplo para exportar
df_ejemplo = pd.DataFrame({
    'año': [2019, 2020, 2021, 2022, 2023],
    'pib': [230000, 225000, 245000, 250000, 258000],
    'inflacion': [2.1, 1.8, 3.5, 8.3, 6.5]
})

# Exportar a CSV
df_ejemplo.to_csv('datos_macro.csv', index=False)
print("Archivo CSV creado: datos_macro.csv")

# Exportar a CSV con opciones
df_ejemplo.to_csv('datos_macro_esp.csv',
                  index=False,
                  sep=';',
                  decimal=',',
                  encoding='utf-8')

# Exportar a Excel
```

```
df_ejemplo.to_excel('datos_macro.xlsx',
                    sheet_name='Datos',
                    index=False)
print("Archivo Excel creado: datos_macro.xlsx")

# Exportar múltiples DataFrames a Excel
# with pd.ExcelWriter('reporte_completo.xlsx') as writer:
#     df_pib.to_excel(writer, sheet_name='PIB', index=False)
#     df_inflacion.to_excel(writer, sheet_name='Inflación', index=False)
#     df_empleo.to_excel(writer, sheet_name='Empleo', index=False)
```

```
Archivo CSV creado: datos_macro.csv
Archivo Excel creado: datos_macro.xlsx
```

3.4 Leer desde otras fuentes

```
# Desde JSON
# df = pd.read_json('datos.json')

# Desde SQL
# import sqlite3
# conn = sqlite3.connect('database.db')
# df = pd.read_sql('SELECT * FROM tabla', conn)

# Desde clipboard (copiar desde Excel)
# df = pd.read_clipboard()

# Desde URL
# url = 'https://datos.gob.pe/dataset/datos.csv'
# df = pd.read_csv(url)
```

4 Selección y filtrado de datos

4.1 Información básica del DataFrame

```
# Crear DataFrame de ejemplo
datos_economicos = {
    'region': ['Lima', 'Arequipa', 'Cusco', 'La Libertad', 'Piura', 'Lambayeque'],
    'pib_millones': [100000, 15000, 8000, 12000, 10000, 7000],
    'poblacion': [9500000, 1300000, 1200000, 1900000, 2000000, 1300000],
    'pobreza': [14.2, 8.5, 12.3, 18.5, 20.1, 15.8],
    'sector_principal': ['Servicios', 'Minería', 'Turismo', 'Agricultura', 'Pesca', '
}

df = pd.DataFrame(datos_economicos)

# Primeras filas
```

```
print("Primeras 3 filas:")
print(df.head(3))

# Últimas filas
print("\nÚltimas 3 filas:")
print(df.tail(3))

# Información general
print("\nInformación del DataFrame:")
print(df.info())

# Estadísticas descriptivas
print("\nEstadísticas descriptivas:")
print(df.describe())

# Columnas
print(f"\nColumnas: {df.columns.tolist()}")

# Forma (filas, columnas)
print(f"Forma: {df.shape}")

# Índice
print(f"\nÍndice: {df.index.tolist()}")
```

```
Primeras 3 filas:
     region  pib_millones  poblacion  pobreza sector_principal
0      Lima        100000    9500000     14.2        Servicios
1  Arequipa         15000    1300000      8.5          Minería
2     Cusco          8000    1200000     12.3          Turismo

Últimas 3 filas:
        region  pib_millones  poblacion  pobreza sector_principal
3  La Libertad         12000    1900000     18.5      Agricultura
4        Piura         10000    2000000     20.1            Pesca
5   Lambayeque          7000    1300000     15.8         Comercio

Información del DataFrame:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6 entries, 0 to 5
Data columns (total 5 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   region            6 non-null      object
 1   pib_millones      6 non-null      int64
 2   poblacion         6 non-null      int64
 3   pobreza           6 non-null      float64
 4   sector_principal  6 non-null      object
dtypes: float64(1), int64(2), object(2)
```

```
memory usage: 372.0+ bytes
None

Estadísticas descriptivas:
        pib_millones     poblacion    pobreza
count       6.000000  6.000000e+00   6.000000
mean    25333.333333  2.866667e+06  14.900000
std     36691.506737  3.267211e+06   4.219479
min      7000.000000  1.200000e+06   8.500000
25%      8500.000000  1.300000e+06  12.775000
50%     11000.000000  1.600000e+06  15.000000
75%     14250.000000  1.975000e+06  17.825000
max    100000.000000  9.500000e+06  20.100000

Columnas: ['region', 'pib_millones', 'poblacion', 'pobreza', 'sector_principal']
Forma: (6, 5)

Índice: [0, 1, 2, 3, 4, 5]
```

4.2 Seleccionar columnas

```python
# Seleccionar una columna (retorna Series)
pib = df['pib_millones']
print("PIB (Series):")
print(type(pib))
print(pib)

# Seleccionar una columna (retorna DataFrame)
pib_df = df[['pib_millones']]
print("\nPIB (DataFrame):")
print(type(pib_df))
print(pib_df)

# Seleccionar múltiples columnas
columnas = ['region', 'pib_millones', 'poblacion']
df_seleccionado = df[columnas]
print("\nColumnas seleccionadas:")
print(df_seleccionado)

# Con notación de punto (solo para nombres sin espacios)
regiones = df.region
print("\nRegiones con notación de punto:")
print(regiones)
```

```
PIB (Series):
<class 'pandas.core.series.Series'>
0    100000
1     15000
```

```
2      8000
3     12000
4     10000
5      7000
Name: pib_millones, dtype: int64

PIB (DataFrame):
<class 'pandas.core.frame.DataFrame'>
   pib_millones
0        100000
1         15000
2          8000
3         12000
4         10000
5          7000

Columnas seleccionadas:
        region  pib_millones  poblacion
0         Lima        100000    9500000
1     Arequipa         15000    1300000
2        Cusco          8000    1200000
3  La Libertad         12000    1900000
4        Piura         10000    2000000
5   Lambayeque          7000    1300000

Regiones con notación de punto:
0           Lima
1       Arequipa
2          Cusco
3    La Libertad
4          Piura
5     Lambayeque
Name: region, dtype: object
```

4.3 Seleccionar filas con loc e iloc

```python
# loc: selección por etiquetas
print("Fila 0 con loc:")
print(df.loc[0])

# Múltiples filas
print("\nFilas 0 a 2:")
print(df.loc[0:2])

# Filas y columnas específicas
print("\nFilas 0-2, columnas específicas:")
print(df.loc[0:2, ['region', 'pib_millones']])
```

```python
# iloc: selección por posición
print("\nPrimeras 3 filas con iloc:")
print(df.iloc[0:3])

# Selección de celdas específicas
print("\nCelda [0, 1]:")
print(df.iloc[0, 1])

# Combinaciones
print("\nFilas 1-3, columnas 0-2:")
print(df.iloc[1:4, 0:3])
```

```
Fila 0 con loc:
region                 Lima
pib_millones         100000
poblacion           9500000
pobreza                14.2
sector_principal    Servicios
Name: 0, dtype: object

Filas 0 a 2:
     region  pib_millones  poblacion  pobreza sector_principal
0      Lima        100000    9500000     14.2        Servicios
1  Arequipa         15000    1300000      8.5          Minería
2     Cusco          8000    1200000     12.3          Turismo

Filas 0-2, columnas específicas:
     region  pib_millones
0      Lima        100000
1  Arequipa         15000
2     Cusco          8000

Primeras 3 filas con iloc:
     region  pib_millones  poblacion  pobreza sector_principal
0      Lima        100000    9500000     14.2        Servicios
1  Arequipa         15000    1300000      8.5          Minería
2     Cusco          8000    1200000     12.3          Turismo

Celda [0, 1]:
100000

Filas 1-3, columnas 0-2:
        region  pib_millones  poblacion
1     Arequipa         15000    1300000
2        Cusco          8000    1200000
3  La Libertad         12000    1900000
```

4.4 Filtrado por condiciones

```
# Filtro simple
regiones_grandes = df[df['poblacion'] > 1500000]
print("Regiones con población > 1.5M:")
print(regiones_grandes)

# Filtro con múltiples condiciones (AND)
economias_grandes = df[(df['pib_millones'] > 10000) & (df['poblacion'] > 1500000)]
print("\nEconomías grandes:")
print(economias_grandes)

# Filtro con múltiples condiciones (OR)
extremos_pobreza = df[(df['pobreza'] < 10) | (df['pobreza'] > 18)]
print("\nExtremos de pobreza:")
print(extremos_pobreza)

# Filtro con isin()
sectores_interes = ['Minería', 'Turismo']
df_sectores = df[df['sector_principal'].isin(sectores_interes)]
print("\nSectores de interés:")
print(df_sectores)

# Filtro con contains (strings)
df_con_a = df[df['region'].str.contains('a', case=False)]
print("\nRegiones que contienen 'a':")
print(df_con_a)

# Filtro negado
no_lima = df[~(df['region'] == 'Lima')]
print("\nRegiones excepto Lima:")
print(no_lima)
```

```
Regiones con población > 1.5M:
        region  pib_millones  poblacion  pobreza sector_principal
0         Lima        100000    9500000     14.2        Servicios
3  La Libertad         12000    1900000     18.5      Agricultura
4        Piura         10000    2000000     20.1            Pesca

Economías grandes:
        region  pib_millones  poblacion  pobreza sector_principal
0         Lima        100000    9500000     14.2        Servicios
3  La Libertad         12000    1900000     18.5      Agricultura

Extremos de pobreza:
        region  pib_millones  poblacion  pobreza sector_principal
1     Arequipa         15000    1300000      8.5          Minería
3  La Libertad         12000    1900000     18.5      Agricultura
```

```
4        Piura         10000    2000000     20.1            Pesca

Sectores de interés:
     region  pib_millones  poblacion  pobreza sector_principal
1  Arequipa         15000    1300000      8.5          Minería
2     Cusco          8000    1200000     12.3          Turismo

Regiones que contienen 'a':
        region  pib_millones  poblacion  pobreza sector_principal
0         Lima        100000    9500000     14.2        Servicios
1     Arequipa         15000    1300000      8.5          Minería
3  La Libertad         12000    1900000     18.5      Agricultura
4        Piura         10000    2000000     20.1            Pesca
5   Lambayeque          7000    1300000     15.8         Comercio

Regiones excepto Lima:
        region  pib_millones  poblacion  pobreza sector_principal
1     Arequipa         15000    1300000      8.5          Minería
2        Cusco          8000    1200000     12.3          Turismo
3  La Libertad         12000    1900000     18.5      Agricultura
4        Piura         10000    2000000     20.1            Pesca
5   Lambayeque          7000    1300000     15.8         Comercio
```

4.5 Query (consultas tipo SQL)

```python
# Query simple
resultado = df.query('pib_millones > 10000')
print("Query: PIB > 10000")
print(resultado)

# Query con múltiples condiciones
resultado = df.query('pib_millones > 10000 and pobreza < 15')
print("\nQuery con AND:")
print(resultado)

# Query con variables
umbral_pib = 10000
resultado = df.query('pib_millones > @umbral_pib')
print("\nQuery con variable externa:")
print(resultado)

# Query con strings
resultado = df.query('sector_principal == "Minería"')
print("\nQuery con string:")
print(resultado)
```

```
Query: PIB > 10000
        region  pib_millones  poblacion  pobreza sector_principal
```

```
0         Lima        100000    9500000     14.2        Servicios
1     Arequipa         15000    1300000      8.5          Minería
3  La Libertad         12000    1900000     18.5      Agricultura

Query con AND:
     region  pib_millones  poblacion  pobreza sector_principal
0      Lima        100000    9500000     14.2        Servicios
1  Arequipa         15000    1300000      8.5          Minería

Query con variable externa:
        region  pib_millones  poblacion  pobreza sector_principal
0         Lima        100000    9500000     14.2        Servicios
1     Arequipa         15000    1300000      8.5          Minería
3  La Libertad         12000    1900000     18.5      Agricultura

Query con string:
     region  pib_millones  poblacion  pobreza sector_principal
1  Arequipa         15000    1300000      8.5          Minería
```

5 Modificación de columnas

5.1 Crear nuevas columnas

```python
# Crear columna calculada
df['pib_per_capita'] = (df['pib_millones'] / df['poblacion']) * 1000000
print("DataFrame con PIB per cápita:")
print(df)

# Columna con operaciones
df['pobreza_categoria'] = df['pobreza'].apply(
    lambda x: 'Alta' if x > 15 else 'Media' if x > 10 else 'Baja'
)
print("\nCon categoría de pobreza:")
print(df[['region', 'pobreza', 'pobreza_categoria']])

# Columna condicional con np.where
df['es_costa'] = np.where(
    df['region'].isin(['Piura', 'La Libertad', 'Lambayeque', 'Lima']),
    'Sí',
    'No'
)
print("\nCon indicador de costa:")
print(df[['region', 'es_costa']])

# Múltiples condiciones con np.select
condiciones = [
    df['pib_millones'] > 50000,
    df['pib_millones'] > 10000,
```

```
    df['pib_millones'] > 5000
]
categorias = ['Muy grande', 'Grande', 'Mediana']

df['tamaño_economia'] = np.select(condiciones, categorias, default='Pequeña')
print("\nCon tamaño de economía:")
print(df[['region', 'pib_millones', 'tamaño_economia']])
```

```
DataFrame con PIB per cápita:
        region  pib_millones  poblacion  pobreza sector_principal  \
0         Lima        100000    9500000     14.2        Servicios
1     Arequipa         15000    1300000      8.5          Minería
2        Cusco          8000    1200000     12.3          Turismo
3  La Libertad         12000    1900000     18.5      Agricultura
4        Piura         10000    2000000     20.1            Pesca
5   Lambayeque          7000    1300000     15.8         Comercio

   pib_per_capita
0    10526.315789
1    11538.461538
2     6666.666667
3     6315.789474
4     5000.000000
5     5384.615385

Con categoría de pobreza:
        region  pobreza pobreza_categoria
0         Lima     14.2             Media
1     Arequipa      8.5              Baja
2        Cusco     12.3             Media
3  La Libertad     18.5              Alta
4        Piura     20.1              Alta
5   Lambayeque     15.8              Alta

Con indicador de costa:
        region es_costa
0         Lima       Sí
1     Arequipa       No
2        Cusco       No
3  La Libertad       Sí
4        Piura       Sí
5   Lambayeque       Sí

Con tamaño de economía:
        region  pib_millones tamaño_economia
0         Lima        100000      Muy grande
1     Arequipa         15000           Grande
2        Cusco          8000         Mediana
```

```
3  La Libertad         12000          Grande
4        Piura         10000         Mediana
5   Lambayeque          7000         Mediana
```

5.2 Modificar columnas existentes

```python
# Modificar valores
df['pib_millones'] = df['pib_millones'] * 1.05  # Incremento 5%
print("PIB incrementado 5%:")
print(df[['region', 'pib_millones']])

# Modificar con condición
df.loc[df['region'] == 'Lima', 'pobreza'] = 13.5
print("\nPobreza de Lima modificada:")
print(df[['region', 'pobreza']])

# Aplicar función
df['region'] = df['region'].str.upper()
print("\nRegiones en mayúsculas:")
print(df['region'])

# Replace
df['sector_principal'] = df['sector_principal'].replace({
    'Servicios': 'Sector Terciario',
    'Minería': 'Sector Primario'
})
print("\nSectores renombrados:")
print(df[['region', 'sector_principal']])
```

```
PIB incrementado 5%:
        region  pib_millones
0         Lima      105000.0
1     Arequipa       15750.0
2        Cusco        8400.0
3  La Libertad       12600.0
4        Piura       10500.0
5   Lambayeque        7350.0

Pobreza de Lima modificada:
        region  pobreza
0         Lima     13.5
1     Arequipa      8.5
2        Cusco     12.3
3  La Libertad     18.5
4        Piura     20.1
5   Lambayeque     15.8

Regiones en mayúsculas:
```

```
0           LIMA
1       AREQUIPA
2          CUSCO
3    LA LIBERTAD
4          PIURA
5     LAMBAYEQUE
Name: region, dtype: object

Sectores renombrados:
        region  sector_principal
0         LIMA  Sector Terciario
1     AREQUIPA   Sector Primario
2        CUSCO           Turismo
3  LA LIBERTAD       Agricultura
4        PIURA             Pesca
5   LAMBAYEQUE          Comercio
```

5.3 Eliminar columnas

```python
# Eliminar una columna
df_sin_columna = df.drop('es_costa', axis=1)
print("Sin columna 'es_costa':")
print(df_sin_columna.columns.tolist())

# Eliminar múltiples columnas
df_reducido = df.drop(['pobreza_categoria', 'tamaño_economia'], axis=1)
print("\nColumnas restantes:")
print(df_reducido.columns.tolist())

# Eliminar in-place (modifica el original)
# df.drop('columna', axis=1, inplace=True)
```

```
Sin columna 'es_costa':
['region', 'pib_millones', 'poblacion', 'pobreza', 'sector_principal', 'pib_per_capit

Columnas restantes:
['region', 'pib_millones', 'poblacion', 'pobreza', 'sector_principal', 'pib_per_capit
```

6 Renombrar columnas

6.1 Métodos de renombrado

```python
# Crear DataFrame de ejemplo
df_original = pd.DataFrame({
    'col1': [1, 2, 3],
    'col2': [4, 5, 6],
    'col3': [7, 8, 9]
})
```

```
# Método 1: rename con diccionario
df_renamed = df_original.rename(columns={
    'col1': 'columna_uno',
    'col2': 'columna_dos'
})
print("Renombrado con rename:")
print(df_renamed)

# Método 2: asignar lista de nombres
df_renamed2 = df_original.copy()
df_renamed2.columns = ['primera', 'segunda', 'tercera']
print("\nRenombrado con lista:")
print(df_renamed2)

# Método 3: modificar todos los nombres
df_renamed3 = df_original.copy()
df_renamed3.columns = df_renamed3.columns.str.upper()
print("\nColumnas en mayúsculas:")
print(df_renamed3)

# Método 4: función para transformar nombres
df_renamed4 = df_original.copy()
df_renamed4.columns = df_renamed4.columns.str.replace('col', 'variable_')
print("\nReemplazo de prefijo:")
print(df_renamed4)
```

```
Renombrado con rename:
   columna_uno  columna_dos  col3
0            1            4     7
1            2            5     8
2            3            6     9

Renombrado con lista:
   primera  segunda  tercera
0        1        4        7
1        2        5        8
2        3        6        9

Columnas en mayúsculas:
   COL1  COL2  COL3
0     1     4     7
1     2     5     8
2     3     6     9

Reemplazo de prefijo:
   variable_1  variable_2  variable_3
0           1           4           7
```

```
1           2           5           8
2           3           6           9
```

6.2 Aplicaciones económicas

```python
# DataFrame con nombres en inglés
df_english = pd.DataFrame({
    'country': ['Peru', 'Chile', 'Argentina'],
    'gdp': [242632, 301000, 487000],
    'population': [33715471, 19493184, 45810000],
    'poverty_rate': [20.5, 10.8, 37.5]
})

# Traducir a español
traducciones = {
    'country': 'pais',
    'gdp': 'pib',
    'population': 'poblacion',
    'poverty_rate': 'tasa_pobreza'
}

df_spanish = df_english.rename(columns=traducciones)
print("Columnas traducidas:")
print(df_spanish)

# Limpiar nombres de columnas problemáticos
df_sucio = pd.DataFrame({
    'Región ': ['Lima', 'Cusco'],
    'PIB (millones)': [100000, 8000],
    'Tasa de Pobreza %': [14.2, 12.3]
})

# Limpiar espacios y caracteres especiales
df_limpio = df_sucio.copy()
df_limpio.columns = (df_limpio.columns
                     .str.strip()  # Eliminar espacios
                     .str.lower()  # A minúsculas
                     .str.replace(' ', '_')  # Espacios a guiones bajos
                     .str.replace('(', '')  # Eliminar paréntesis
                     .str.replace(')', '')
                     .str.replace('%', 'porcentaje'))

print("\nColumnas limpias:")
print(df_limpio)
```

```
Columnas traducidas:
        pais     pib  poblacion  tasa_pobreza
0       Peru  242632   33715471          20.5
```

```
1      Chile  301000   19493184          10.8
2  Argentina  487000   45810000          37.5

Columnas limpias:
  región  pib_millones  tasa_de_pobreza_porcentaje
0   Lima        100000                        14.2
1  Cusco          8000                        12.3
```

7 Operaciones con DataFrames

7.1 Operaciones aritméticas

```
# Crear DataFrame de ventas
ventas = pd.DataFrame({
    'producto': ['A', 'B', 'C', 'D'],
    'ene': [1000, 1500, 800, 1200],
    'feb': [1100, 1400, 850, 1300],
    'mar': [1200, 1600, 900, 1250]
})

print("Ventas originales:")
print(ventas)

# Sumar columnas
ventas['total'] = ventas['ene'] + ventas['feb'] + ventas['mar']
print("\nCon total:")
print(ventas)

# Promedio
ventas['promedio'] = ventas[['ene', 'feb', 'mar']].mean(axis=1)
print("\nCon promedio:")
print(ventas)

# Operaciones por fila
ventas['total_sum'] = ventas[['ene', 'feb', 'mar']].sum(axis=1)
ventas['max_mes'] = ventas[['ene', 'feb', 'mar']].max(axis=1)
ventas['min_mes'] = ventas[['ene', 'feb', 'mar']].min(axis=1)

print("\nEstadísticas por fila:")
print(ventas)
```

```
Ventas originales:
  producto   ene   feb   mar
0        A  1000  1100  1200
1        B  1500  1400  1600
2        C   800   850   900
3        D  1200  1300  1250
```

```
Con total:
  producto   ene   feb   mar  total
0        A  1000  1100  1200   3300
1        B  1500  1400  1600   4500
2        C   800   850   900   2550
3        D  1200  1300  1250   3750

Con promedio:
  producto   ene   feb   mar  total  promedio
0        A  1000  1100  1200   3300    1100.0
1        B  1500  1400  1600   4500    1500.0
2        C   800   850   900   2550     850.0
3        D  1200  1300  1250   3750    1250.0

Estadísticas por fila:
  producto   ene   feb   mar  total  promedio  total_sum  max_mes  min_mes
0        A  1000  1100  1200   3300    1100.0       3300     1200     1000
1        B  1500  1400  1600   4500    1500.0       4500     1600     1400
2        C   800   850   900   2550     850.0       2550      900      800
3        D  1200  1300  1250   3750    1250.0       3750     1300     1200
```

7.2 Aplicar funciones

```python
# apply(): aplicar función a columnas o filas
def calcular_igv(monto):
    return monto * 0.18

ventas['igv_ene'] = ventas['ene'].apply(calcular_igv)
print("\nCon IGV de enero:")
print(ventas[['producto', 'ene', 'igv_ene']])

# Lambda functions
ventas['ene_con_igv'] = ventas['ene'].apply(lambda x: x * 1.18)
print("\nEnero con IGV:")
print(ventas[['producto', 'ene', 'ene_con_igv']])

# apply() a múltiples columnas
def calcular_variacion(row):
    return ((row['mar'] - row['ene']) / row['ene']) * 100

ventas['variacion_porcentual'] = ventas.apply(calcular_variacion, axis=1)
print("\nCon variación porcentual:")
print(ventas[['producto', 'ene', 'mar', 'variacion_porcentual']])

# applymap(): aplicar función a todo el DataFrame
df_numerico = ventas[['ene', 'feb', 'mar']].applymap(lambda x: x * 1.18)
print("\nTodo con IGV:")
print(df_numerico)
```

```
Con IGV de enero:
  producto   ene  igv_ene
0        A  1000    180.0
1        B  1500    270.0
2        C   800    144.0
3        D  1200    216.0

Enero con IGV:
  producto   ene  ene_con_igv
0        A  1000       1180.0
1        B  1500       1770.0
2        C   800        944.0
3        D  1200       1416.0

Con variación porcentual:
  producto   ene   mar  variacion_porcentual
0        A  1000  1200             20.000000
1        B  1500  1600              6.666667
2        C   800   900             12.500000
3        D  1200  1250              4.166667

Todo con IGV:
      ene     feb     mar
0  1180.0  1298.0  1416.0
1  1770.0  1652.0  1888.0
2   944.0  1003.0  1062.0
3  1416.0  1534.0  1475.0
```

7.3 Operaciones de agregación

```python
# Crear datos de ventas por sucursal
datos_sucursales = {
    'sucursal': ['Lima', 'Lima', 'Lima', 'Arequipa', 'Arequipa', 'Cusco'],
    'producto': ['A', 'B', 'C', 'A', 'B', 'A'],
    'ventas': [10000, 15000, 8000, 12000, 9000, 7000],
    'unidades': [100, 150, 80, 120, 90, 70]
}

df_sucursales = pd.DataFrame(datos_sucursales)
print("Ventas por sucursal:")
print(df_sucursales)

# Suma total
print(f"\nVentas totales: S/ {df_sucursales['ventas'].sum():,}")

# Estadísticas básicas
print(f"Promedio ventas: S/ {df_sucursales['ventas'].mean():,.2f}")
print(f"Mediana ventas: S/ {df_sucursales['ventas'].median():,.2f}")
```

```
print(f"Desviación estándar: S/ {df_sucursales['ventas'].std():,.2f}")
print(f"Mínimo: S/ {df_sucursales['ventas'].min():,}")
print(f"Máximo: S/ {df_sucursales['ventas'].max():,}")

# value_counts(): contar ocurrencias
print("\nProductos más vendidos:")
print(df_sucursales['producto'].value_counts())

print("\nVentas por sucursal:")
print(df_sucursales.groupby('sucursal')['ventas'].sum())
```

```
Ventas por sucursal:
   sucursal producto  ventas  unidades
0      Lima        A   10000       100
1      Lima        B   15000       150
2      Lima        C    8000        80
3  Arequipa        A   12000       120
4  Arequipa        B    9000        90
5     Cusco        A    7000        70

Ventas totales: S/ 61,000
Promedio ventas: S/ 10,166.67
Mediana ventas: S/ 9,500.00
Desviación estándar: S/ 2,926.89
Mínimo: S/ 7,000
Máximo: S/ 15,000

Productos más vendidos:
producto
A    3
B    2
C    1
Name: count, dtype: int64

Ventas por sucursal:
sucursal
Arequipa    21000
Cusco        7000
Lima        33000
Name: ventas, dtype: int64
```

8 Merge y Join

El merge es una de las operaciones más importantes en análisis de datos, permite combinar DataFrames.

8.1 Tipos de merge

```
# Crear DataFrames de ejemplo
df_ventas = pd.DataFrame({
    'producto_id': ['P001', 'P002', 'P003', 'P004'],
    'ventas': [10000, 15000, 8000, 12000],
    'unidades': [100, 150, 80, 120]
})

df_productos = pd.DataFrame({
    'producto_id': ['P001', 'P002', 'P003', 'P005'],
    'nombre': ['Laptop', 'Mouse', 'Teclado', 'Monitor'],
    'precio': [2500, 25, 80, 800]
})

print("DataFrame de ventas:")
print(df_ventas)
print("\nDataFrame de productos:")
print(df_productos)

# Inner join (solo coincidencias)
df_inner = pd.merge(df_ventas, df_productos, on='producto_id', how='inner')
print("\nInner join:")
print(df_inner)

# Left join (todas las ventas)
df_left = pd.merge(df_ventas, df_productos, on='producto_id', how='left')
print("\nLeft join:")
print(df_left)

# Right join (todos los productos)
df_right = pd.merge(df_ventas, df_productos, on='producto_id', how='right')
print("\nRight join:")
print(df_right)

# Outer join (todos)
df_outer = pd.merge(df_ventas, df_productos, on='producto_id', how='outer')
print("\nOuter join:")
print(df_outer)
```

```
DataFrame de ventas:
  producto_id  ventas  unidades
0        P001   10000       100
1        P002   15000       150
2        P003    8000        80
3        P004   12000       120

DataFrame de productos:
```

```
  producto_id   nombre  precio
0        P001   Laptop    2500
1        P002    Mouse      25
2        P003  Teclado      80
3        P005  Monitor     800

Inner join:
  producto_id  ventas  unidades   nombre  precio
0        P001   10000       100   Laptop    2500
1        P002   15000       150    Mouse      25
2        P003    8000        80  Teclado      80

Left join:
  producto_id  ventas  unidades   nombre  precio
0        P001   10000       100   Laptop  2500.0
1        P002   15000       150    Mouse    25.0
2        P003    8000        80  Teclado    80.0
3        P004   12000       120      NaN     NaN

Right join:
  producto_id   ventas  unidades   nombre  precio
0        P001  10000.0     100.0   Laptop    2500
1        P002  15000.0     150.0    Mouse      25
2        P003   8000.0      80.0  Teclado      80
3        P005      NaN       NaN  Monitor     800

Outer join:
  producto_id   ventas  unidades   nombre  precio
0        P001  10000.0     100.0   Laptop  2500.0
1        P002  15000.0     150.0    Mouse    25.0
2        P003   8000.0      80.0  Teclado    80.0
3        P004  12000.0     120.0      NaN     NaN
4        P005      NaN       NaN  Monitor   800.0
```

8.2 Merge con indicador

```python
# Merge con indicador para rastrear coincidencias
df_merged = pd.merge(
    df_ventas,
    df_productos,
    on='producto_id',
    how='outer',
    indicator=True
)

print("Merge con indicador:")
print(df_merged)
```

```
# Analizar coincidencias
print("\nAnálisis de merge:")
print(df_merged['_merge'].value_counts())

# Filtrar por tipo de coincidencia
solo_ventas = df_merged[df_merged['_merge'] == 'left_only']
print("\nProductos solo en ventas:")
print(solo_ventas)
```

```
Merge con indicador:
  producto_id   ventas  unidades   nombre  precio      _merge
0        P001  10000.0     100.0   Laptop  2500.0        both
1        P002  15000.0     150.0    Mouse    25.0        both
2        P003   8000.0      80.0  Teclado    80.0        both
3        P004  12000.0     120.0      NaN     NaN   left_only
4        P005      NaN       NaN  Monitor   800.0  right_only

Análisis de merge:
_merge
both          3
left_only     1
right_only    1
Name: count, dtype: int64

Productos solo en ventas:
  producto_id   ventas  unidades nombre  precio     _merge
3        P004  12000.0     120.0    NaN     NaN  left_only
```

8.3 Merge con diferentes nombres de columna

```
# DataFrames con diferentes nombres de clave
df1 = pd.DataFrame({
    'cliente_id': [1, 2, 3],
    'ventas': [1000, 1500, 800]
})

df2 = pd.DataFrame({
    'id': [1, 2, 4],
    'nombre': ['Juan', 'María', 'Pedro']
})

# Merge especificando columnas diferentes
df_merged = pd.merge(
    df1,
    df2,
    left_on='cliente_id',
    right_on='id',
```

```
    how='left'
)

print("Merge con diferentes nombres:")
print(df_merged)
```

```
Merge con diferentes nombres:
   cliente_id  ventas   id nombre
0           1    1000  1.0   Juan
1           2    1500  2.0  María
2           3     800  NaN    NaN
```

8.4 Merge múltiple

```
# Merge de tres DataFrames
df_ventas = pd.DataFrame({
    'venta_id': [1, 2, 3],
    'producto_id': ['P001', 'P002', 'P001'],
    'monto': [2500, 50, 2500]
})

df_productos = pd.DataFrame({
    'producto_id': ['P001', 'P002'],
    'categoria_id': ['C001', 'C002']
})

df_categorias = pd.DataFrame({
    'categoria_id': ['C001', 'C002'],
    'categoria': ['Computadoras', 'Accesorios']
})

# Primer merge
df_temp = pd.merge(df_ventas, df_productos, on='producto_id')
# Segundo merge
df_completo = pd.merge(df_temp, df_categorias, on='categoria_id')

print("Merge de tres DataFrames:")
print(df_completo)
```

```
Merge de tres DataFrames:
   venta_id producto_id  monto categoria_id     categoria
0         1        P001   2500         C001  Computadoras
1         2        P002     50         C002    Accesorios
2         3        P001   2500         C001  Computadoras
```

9 Append y Concatenación

9.1 Append (agregar filas)

```
# Crear DataFrames para concatenar
df1 = pd.DataFrame({
    'mes': ['Enero', 'Febrero'],
    'ventas': [10000, 11000]
})

df2 = pd.DataFrame({
    'mes': ['Marzo', 'Abril'],
    'ventas': [12000, 11500]
})

# Append (deprecado, usar concat)
# df_append = df1.append(df2, ignore_index=True)

# Método recomendado: concat
df_concatenado = pd.concat([df1, df2], ignore_index=True)
print("DataFrames concatenados:")
print(df_concatenado)
```

```
DataFrames concatenados:
       mes  ventas
0    Enero   10000
1  Febrero   11000
2    Marzo   12000
3    Abril   11500
```

9.2 Concatenación vertical y horizontal

```
# Concatenación vertical (apilar filas)
df_2022 = pd.DataFrame({
    'trimestre': ['Q1', 'Q2', 'Q3', 'Q4'],
    'pib': [50000, 51000, 52000, 53000]
})

df_2023 = pd.DataFrame({
    'trimestre': ['Q1', 'Q2', 'Q3', 'Q4'],
    'pib': [54000, 55000, 56000, 57000]
})

df_vertical = pd.concat([df_2022, df_2023], ignore_index=True)
print("Concatenación vertical:")
print(df_vertical)
```

```python
# Añadir etiqueta de año
df_2022['año'] = 2022
df_2023['año'] = 2023
df_con_año = pd.concat([df_2022, df_2023], ignore_index=True)
print("\nCon identificador de año:")
print(df_con_año)

# Concatenación horizontal (agregar columnas)
df_ingresos = pd.DataFrame({
    'mes': ['Ene', 'Feb', 'Mar'],
    'ingresos': [10000, 11000, 12000]
})

df_gastos = pd.DataFrame({
    'gastos': [7000, 7500, 8000],
    'utilidad': [3000, 3500, 4000]
})

df_horizontal = pd.concat([df_ingresos, df_gastos], axis=1)
print("\nConcatenación horizontal:")
print(df_horizontal)
```

```
Concatenación vertical:
  trimestre    pib
0        Q1  50000
1        Q2  51000
2        Q3  52000
3        Q4  53000
4        Q1  54000
5        Q2  55000
6        Q3  56000
7        Q4  57000

Con identificador de año:
  trimestre    pib   año
0        Q1  50000  2022
1        Q2  51000  2022
2        Q3  52000  2022
3        Q4  53000  2022
4        Q1  54000  2023
5        Q2  55000  2023
6        Q3  56000  2023
7        Q4  57000  2023

Concatenación horizontal:
   mes  ingresos  gastos  utilidad
0  Ene     10000    7000      3000
1  Feb     11000    7500      3500
```

```
2  Mar      12000     8000       4000
```

10 Agrupación y agregación

10.1 GroupBy básico

```python
# Crear datos de ventas por región y producto
datos = {
    'region': ['Lima', 'Lima', 'Lima', 'Arequipa', 'Arequipa', 'Cusco', 'Cusco'],
    'producto': ['A', 'B', 'A', 'A', 'B', 'A', 'B'],
    'ventas': [10000, 15000, 11000, 8000, 9000, 7000, 6500],
    'unidades': [100, 150, 110, 80, 90, 70, 65]
}

df = pd.DataFrame(datos)
print("Datos originales:")
print(df)

# Agrupar y sumar
ventas_por_region = df.groupby('region')['ventas'].sum()
print("\nVentas por región:")
print(ventas_por_region)

# Múltiples agregaciones
resumen = df.groupby('region').agg({
    'ventas': ['sum', 'mean', 'count'],
    'unidades': ['sum', 'mean']
})
print("\nResumen por región:")
print(resumen)
```

```
Datos originales:
     region producto  ventas  unidades
0      Lima        A   10000       100
1      Lima        B   15000       150
2      Lima        A   11000       110
3  Arequipa        A    8000        80
4  Arequipa        B    9000        90
5     Cusco        A    7000        70
6     Cusco        B    6500        65

Ventas por región:
region
Arequipa    17000
Cusco       13500
Lima        36000
Name: ventas, dtype: int64
```

```
Resumen por región:
         ventas                 unidades
            sum     mean count      sum   mean
region
Arequipa  17000   8500.0     2      170   85.0
Cusco     13500   6750.0     2      135   67.5
Lima      36000  12000.0     3      360  120.0
```

10.2 GroupBy con múltiples columnas

```
# Agrupar por región y producto
ventas_region_producto = df.groupby(['region', 'producto'])['ventas'].sum()
print("Ventas por región y producto:")
print(ventas_region_producto)

# Desagrupar (unstack)
tabla_pivot = ventas_region_producto.unstack(fill_value=0)
print("\nTabla pivoteable:")
print(tabla_pivot)

# Agregaciones personalizadas
resumen_completo = df.groupby(['region', 'producto']).agg({
    'ventas': ['sum', 'mean'],
    'unidades': 'sum'
}).round(2)

print("\nResumen completo:")
print(resumen_completo)
```

```
Ventas por región y producto:
region    producto
Arequipa  A            8000
          B            9000
Cusco     A            7000
          B            6500
Lima      A           21000
          B           15000
Name: ventas, dtype: int64

Tabla pivoteable:
producto      A      B
region
Arequipa   8000   9000
Cusco      7000   6500
Lima      21000  15000

Resumen completo:
                 ventas          unidades
```

```
                    sum     mean      sum
region   producto
Arequipa A         8000   8000.0       80
         B         9000   9000.0       90
Cusco    A         7000   7000.0       70
         B         6500   6500.0       65
Lima     A        21000  10500.0      210
         B        15000  15000.0      150
```

10.3 Funciones de agregación personalizadas

```python
# Definir función personalizada
def rango(x):
    return x.max() - x.min()

# Aplicar múltiples funciones
resumen = df.groupby('region')['ventas'].agg([
    'count',
    'sum',
    'mean',
    'std',
    'min',
    'max',
    rango
])

print("Resumen con función personalizada:")
print(resumen)

# Transform: mantener tamaño original
df['ventas_region_promedio'] = df.groupby('region')['ventas'].transform('mean')
df['ventas_vs_promedio'] = df['ventas'] - df['ventas_region_promedio']

print("\nCon comparación vs promedio regional:")
print(df)
```

```
Resumen con función personalizada:
          count    sum     mean          std    min    max  rango
region
Arequipa      2  17000   8500.0   707.106781   8000   9000   1000
Cusco         2  13500   6750.0   353.553391   6500   7000    500
Lima          3  36000  12000.0  2645.751311  10000  15000   5000

Con comparación vs promedio regional:
     region producto  ventas  unidades  ventas_region_promedio  \
0      Lima        A   10000       100                 12000.0
1      Lima        B   15000       150                 12000.0
2      Lima        A   11000       110                 12000.0
```

```
3  Arequipa        A    8000        80                8500.0
4  Arequipa        B    9000        90                8500.0
5     Cusco        A    7000        70                6750.0
6     Cusco        B    6500        65                6750.0

   ventas_vs_promedio
0             -2000.0
1              3000.0
2             -1000.0
3              -500.0
4               500.0
5               250.0
6              -250.0
```

11 Aplicaciones económicas

11.1 Aplicación 1: Análisis de datos COVID-19

```python
# Simular datos de COVID-19 por región
np.random.seed(42)

regiones = ['Lima', 'Arequipa', 'Cusco', 'La Libertad', 'Piura'] * 20
fechas = pd.date_range('2020-03-01', periods=100, freq='D')

df_covid = pd.DataFrame({
    'fecha': np.random.choice(fechas, 100),
    'region': regiones,
    'casos_positivos': np.random.randint(10, 500, 100),
    'fallecidos': np.random.randint(0, 20, 100),
    'pruebas_realizadas': np.random.randint(50, 1000, 100),
    'grupo_edad': np.random.choice(['0-19', '20-44', '45-64', '65+'], 100)
})

# Agregar datos demográficos
df_poblacion = pd.DataFrame({
    'region': ['Lima', 'Arequipa', 'Cusco', 'La Libertad', 'Piura'],
    'poblacion': [9500000, 1300000, 1200000, 1900000, 2000000]
})

# Merge de datos
df_analisis = pd.merge(df_covid, df_poblacion, on='region')

# Calcular métricas
df_analisis['tasa_positividad'] = (
    df_analisis['casos_positivos'] / df_analisis['pruebas_realizadas'] * 100
)

df_analisis['casos_por_100k'] = (
```

```
    df_analisis['casos_positivos'] / df_analisis['poblacion'] * 100000
)

# Análisis por región
print("ANÁLISIS COVID-19 POR REGIÓN")
print("=" * 70)

resumen_regional = df_analisis.groupby('region').agg({
    'casos_positivos': 'sum',
    'fallecidos': 'sum',
    'pruebas_realizadas': 'sum',
    'tasa_positividad': 'mean',
    'casos_por_100k': 'sum'
}).round(2)

resumen_regional['tasa_letalidad'] = (
    resumen_regional['fallecidos'] / resumen_regional['casos_positivos'] * 100
).round(2)

print(resumen_regional)

# Análisis por grupo de edad
print("\n\nANÁLISIS POR GRUPO DE EDAD")
print("=" * 70)

resumen_edad = df_analisis.groupby('grupo_edad').agg({
    'casos_positivos': 'sum',
    'fallecidos': 'sum'
})

resumen_edad['tasa_letalidad'] = (
    resumen_edad['fallecidos'] / resumen_edad['casos_positivos'] * 100
).round(2)

print(resumen_edad.sort_values('tasa_letalidad', ascending=False))

# Serie de tiempo
df_serie = df_analisis.groupby('fecha').agg({
    'casos_positivos': 'sum',
    'fallecidos': 'sum'
}).sort_index()

# Promedio móvil de 7 días
df_serie['casos_ma7'] = df_serie['casos_positivos'].rolling(window=7).mean()

print("\n\nSERIE DE TIEMPO (últimos 10 días)")
print("=" * 70)
```

```
print(df_serie.tail(10).round(1))
```

```
ANÁLISIS COVID-19 POR REGIÓN
======================================================================
             casos_positivos  fallecidos  pruebas_realizadas  \
region
Arequipa                5198         251                8025
Cusco                   4377         167                8095
La Libertad             4103         169               12590
Lima                    5604         143                9849
Piura                   3767         188               10552

             tasa_positividad  casos_por_100k  tasa_letalidad
region
Arequipa                76.36          399.85            4.83
Cusco                  116.39          364.75            3.82
La Libertad             40.97          215.95            4.12
Lima                    98.20           58.99            2.55
Piura                   75.28          188.35            4.99


ANÁLISIS POR GRUPO DE EDAD
======================================================================
            casos_positivos  fallecidos  tasa_letalidad
grupo_edad
45-64                  4839         208            4.30
65+                    6330         254            4.01
20-44                  7402         285            3.85
0-19                   4478         171            3.82


SERIE DE TIEMPO (últimos 10 días)
======================================================================
            casos_positivos  fallecidos  casos_ma7
fecha
2020-05-24              450           2      245.9
2020-05-26              203          30      237.3
2020-05-27              426          15      271.7
2020-05-28              190          17      288.6
2020-05-29              416          20      306.1
2020-05-30              144           4      303.0
2020-05-31              679          19      358.3
2020-06-01              642          21      385.7
2020-06-03              196           4      384.7
2020-06-08              505          20      396.0
```

11.2 Aplicación 2: Análisis de comercio internacional

```
# Simular datos de exportaciones
productos = ['Cobre', 'Oro', 'Zinc', 'Plomo', 'Plata', 'Café', 'Espárragos']
paises = ['China', 'USA', 'Canadá', 'Japón', 'Corea del Sur', 'Brasil']
años = [2019, 2020, 2021, 2022, 2023]

np.random.seed(42)

n_registros = 500

df_exportaciones = pd.DataFrame({
    'año': np.random.choice(años, n_registros),
    'mes': np.random.randint(1, 13, n_registros),
    'producto': np.random.choice(productos, n_registros),
    'pais_destino': np.random.choice(paises, n_registros),
    'valor_fob_usd': np.random.uniform(100000, 10000000, n_registros),
    'peso_kg': np.random.uniform(1000, 100000, n_registros)
})

# Calcular precio por kg
df_exportaciones['precio_kg'] = (
    df_exportaciones['valor_fob_usd'] / df_exportaciones['peso_kg']
)

print("ANÁLISIS DE EXPORTACIONES")
print("=" * 70)

# Top 10 exportaciones por valor
print("\nTop 10 exportaciones individuales:")
top10 = df_exportaciones.nlargest(10, 'valor_fob_usd')[
    ['año', 'mes', 'producto', 'pais_destino', 'valor_fob_usd']
]
print(top10.to_string(index=False))

# Exportaciones por producto
print("\n\nEXPORTACIONES POR PRODUCTO")
print("=" * 70)

por_producto = df_exportaciones.groupby('producto').agg({
    'valor_fob_usd': ['sum', 'mean', 'count'],
    'peso_kg': 'sum',
    'precio_kg': 'mean'
}).round(2)

por_producto.columns = ['_'.join(col).strip() for col in por_producto.columns]
por_producto = por_producto.sort_values('valor_fob_usd_sum', ascending=False)
print(por_producto)
```

```
# Exportaciones por destino
print("\n\nEXPORTACIONES POR PAÍS DESTINO")
print("=" * 70)

por_pais = df_exportaciones.groupby('pais_destino').agg({
    'valor_fob_usd': 'sum',
    'producto': lambda x: x.value_counts().index[0]  # Producto más exportado
}).round(2)

por_pais.columns = ['valor_total_usd', 'producto_principal']
por_pais = por_pais.sort_values('valor_total_usd', ascending=False)
print(por_pais)

# Evolución anual
print("\n\nEVOLUCIÓN ANUAL DE EXPORTACIONES")
print("=" * 70)

evolucion = df_exportaciones.groupby('año')['valor_fob_usd'].sum()
evolucion_pct = evolucion.pct_change() * 100

resumen_anual = pd.DataFrame({
    'exportaciones_usd': evolucion,
    'variacion_pct': evolucion_pct
}).round(2)

print(resumen_anual)

# Matriz de exportaciones: producto x destino
print("\n\nMATRIZ: PRODUCTO X DESTINO (Millones USD)")
print("=" * 70)

matriz = df_exportaciones.pivot_table(
    values='valor_fob_usd',
    index='producto',
    columns='pais_destino',
    aggfunc='sum',
    fill_value=0
) / 1000000  # Convertir a millones

print(matriz.round(2))
```

```
ANÁLISIS DE EXPORTACIONES
======================================================================

Top 10 exportaciones individuales:
 año  mes   producto  pais_destino  valor_fob_usd
2022    5      Plomo           USA   9.955257e+06
```

```
2023    7 Espárragos         China   9.934036e+06
2020   12       Café        Brasil   9.922120e+06
2020    2      Plomo        Canadá   9.916725e+06
2021    4      Cobre        Canadá   9.883925e+06
2020    4 Espárragos        Brasil   9.868261e+06
2023    4 Espárragos        Canadá   9.814240e+06
2022   12      Plomo        Brasil   9.788450e+06
2019    7 Espárragos        Brasil   9.769580e+06
2023    5      Cobre Corea del Sur   9.762309e+06


EXPORTACIONES POR PRODUCTO
======================================================================
            valor_fob_usd_sum  valor_fob_usd_mean  valor_fob_usd_count  \
producto
Cobre            3.884783e+08          4980491.61                   78
Espárragos       3.699379e+08          4932504.79                   75
Café             3.666037e+08          5163432.90                   71
Zinc             3.558708e+08          4809065.36                   74
Plomo            3.343797e+08          5572994.53                   60
Oro              3.316140e+08          4605749.81                   72
Plata            3.174952e+08          4535645.48                   70

            peso_kg_sum  precio_kg_mean
producto
Cobre        4415310.17          190.40
Espárragos   3851545.49          224.55
Café         3791869.02          328.37
Zinc         3152074.23          213.16
Plomo        2965285.40          300.92
Oro          3146402.23          277.06
Plata        3811206.16          187.43


EXPORTACIONES POR PAÍS DESTINO
======================================================================
               valor_total_usd producto_principal
pais_destino
USA               4.477071e+08               Oro
Canadá            4.214130e+08              Zinc
Corea del Sur     4.208985e+08             Plomo
Brasil            4.052131e+08             Cobre
Japón             3.852828e+08             Cobre
China             3.838651e+08              Zinc


EVOLUCIÓN ANUAL DE EXPORTACIONES
======================================================================
```

```
      exportaciones_usd  variacion_pct
año
2019       5.416679e+08            NaN
2020       4.880700e+08          -9.89
2021       4.630620e+08          -5.12
2022       5.181818e+08          11.90
2023       4.533979e+08         -12.50


MATRIZ: PRODUCTO X DESTINO (Millones USD)
======================================================================
pais_destino  Brasil  Canadá  China  Corea del Sur  Japón    USA
producto
Café           86.57   49.45  69.19          50.29  39.77  71.33
Cobre          66.51   72.20  46.98          57.29  70.67  74.83
Espárragos     69.00   62.05  40.97          66.20  63.74  67.99
Oro            55.50   46.88  50.55          54.74  45.73  78.23
Plata          38.38   53.34  57.31          65.20  31.18  72.10
Plomo          51.73   45.89  46.61          82.74  57.60  49.82
Zinc           37.53   91.61  72.26          44.45  76.60  33.42
```

12 Ejercicios prácticos

12.1 Ejercicio 1: Análisis de indicadores macroeconómicos

```python
# Crear dataset de indicadores macroeconómicos
años = list(range(2010, 2024))
n_años = len(años)

df_macro = pd.DataFrame({
    'año': años,
    'pib_millones': [190000, 200000, 208000, 215000, 210000, 220000,
                     225000, 218000, 225000, 235000, 220000, 240000,
                     250000, 258000],
    'inflacion': [2.1, 2.5, 2.8, 2.9, 3.2, 3.5, 3.2, 2.8, 1.8, 2.1,
                  2.5, 3.5, 8.3, 6.5],
    'desempleo': [7.8, 7.5, 6.8, 6.5, 6.2, 6.7, 6.9, 7.2, 13.0, 11.9,
                  9.8, 7.8, 7.2, 7.5],
    'tipo_cambio': [2.80, 2.82, 2.79, 2.84, 3.18, 3.38, 3.37, 3.36,
                    3.49, 3.88, 3.83, 3.82, 3.75, 3.72],
    'exportaciones': [35000, 46000, 45600, 42800, 34200, 34000,
                      37000, 44900, 42900, 47600, 42900, 63200,
                      68000, 72000],
    'importaciones': [28800, 37000, 41000, 42300, 41100, 37000,
                      35000, 38700, 33900, 36000, 35600, 53800,
                      58000, 62000]
})
```

```
print("ANÁLISIS DE INDICADORES MACROECONÓMICOS")
print("=" * 70)

# Calcular indicadores derivados
df_macro['crecimiento_pib'] = df_macro['pib_millones'].pct_change() * 100
df_macro['balanza_comercial'] = df_macro['exportaciones'] - df_macro['importaciones']
df_macro['coeficiente_apertura'] = (
    (df_macro['exportaciones'] + df_macro['importaciones']) /
    df_macro['pib_millones'] * 100
)

# Estadísticas descriptivas
print("\nEstadísticas descriptivas:")
print(df_macro[['crecimiento_pib', 'inflacion', 'desempleo']].describe().round(2))

# Años con mayor/menor crecimiento
print("\n\nAños con mayor crecimiento:")
print(df_macro.nlargest(3, 'crecimiento_pib')[['año', 'crecimiento_pib', 'pib_millone

print("\nAños con menor crecimiento:")
print(df_macro.nsmallest(3, 'crecimiento_pib')[['año', 'crecimiento_pib', 'pib_millon

# Correlaciones
print("\n\nCorrelación entre variables:")
correlaciones = df_macro[['crecimiento_pib', 'inflacion', 'desempleo',
                          'balanza_comercial']].corr().round(3)
print(correlaciones)

# Períodos de crisis (crecimiento negativo)
crisis = df_macro[df_macro['crecimiento_pib'] < 0]
print(f"\n\nPeríodos de contracción económica:")
print(crisis[['año', 'crecimiento_pib', 'desempleo', 'inflacion']])

# Promedios por década
df_macro['decada'] = (df_macro['año'] // 10) * 10
promedios_decada = df_macro.groupby('decada')[
    ['crecimiento_pib', 'inflacion', 'desempleo']
].mean().round(2)

print("\n\nPromedios por década:")
print(promedios_decada)

# Guardar resultados
df_macro.to_csv('analisis_macroeconomico.csv', index=False)
print("\n\nResultados guardados en 'analisis_macroeconomico.csv'")
```

```
ANÁLISIS DE INDICADORES MACROECONÓMICOS
======================================================================
```

```
Estadísticas descriptivas:
       crecimiento_pib  inflacion  desempleo
count            13.00      14.00      14.00
mean              2.46       3.41       8.06
std               4.09       1.80       2.06
min              -6.38       1.80       6.20
25%               2.27       2.50       6.82
50%               3.37       2.85       7.35
75%               4.44       3.42       7.80
max               9.09       8.30      13.00


Años con mayor crecimiento:
     año  crecimiento_pib  pib_millones
11  2021         9.090909        240000
1   2011         5.263158        200000
5   2015         4.761905        220000

Años con menor crecimiento:
     año  crecimiento_pib  pib_millones
10  2020        -6.382979        220000
7   2017        -3.111111        218000
4   2014        -2.325581        210000


Correlación entre variables:
                   crecimiento_pib  inflacion  desempleo  balanza_comercial
crecimiento_pib              1.000      0.180     -
0.017              0.274
inflacion                    0.180      1.000     -
0.354              0.173
desempleo                   -0.017     -0.354      1.000              0.558
balanza_comercial            0.274      0.173      0.558              1.000


Períodos de contracción económica:
     año  crecimiento_pib  desempleo  inflacion
4   2014        -2.325581        6.2        3.2
7   2017        -3.111111        7.2        2.8
10  2020        -6.382979        9.8        2.5


Promedios por década:
        crecimiento_pib  inflacion  desempleo
decada
2010               2.43       2.69       8.05
2020               2.52       5.20       8.07
```

```
Resultados guardados en 'analisis_macroeconomico.csv'
```

12.2 Ejercicio 2: Sistema de gestión de inventarios

```python
# Crear sistema de inventarios
np.random.seed(42)

# Catálogo de productos
df_productos = pd.DataFrame({
    'producto_id': ['PROD' + str(i).zfill(3) for i in range(1, 21)],
    'nombre': [f'Producto {i}' for i in range(1, 21)],
    'categoria': np.random.choice(['Electrónica', 'Ropa', 'Alimentos', 'Hogar'], 20),
    'precio_costo': np.random.uniform(10, 500, 20).round(2),
    'precio_venta': np.random.uniform(15, 750, 20).round(2),
    'stock_actual': np.random.randint(0, 200, 20),
    'stock_minimo': np.random.randint(10, 50, 20)
})

# Ajustar precio_venta para que sea mayor que costo
df_productos['precio_venta'] = (df_productos['precio_costo'] *
                                np.random.uniform(1.2, 2.0, 20)).round(2)

# Transacciones de ventas
n_transacciones = 500
fechas = pd.date_range('2023-01-01', '2023-12-31', freq='D')

df_ventas = pd.DataFrame({
    'fecha': np.random.choice(fechas, n_transacciones),
    'producto_id': np.random.choice(df_productos['producto_id'], n_transacciones),
    'cantidad': np.random.randint(1, 20, n_transacciones),
    'tipo': 'venta'
})

# Transacciones de compras (reposición)
df_compras = pd.DataFrame({
    'fecha': np.random.choice(fechas, 100),
    'producto_id': np.random.choice(df_productos['producto_id'], 100),
    'cantidad': np.random.randint(10, 100, 100),
    'tipo': 'compra'
})

# Combinar transacciones
df_transacciones = pd.concat([df_ventas, df_compras], ignore_index=True)
df_transacciones = df_transacciones.sort_values('fecha').reset_index(drop=True)

# Merge con información de productos
```

```
df_transacciones = pd.merge(
    df_transacciones,
    df_productos[['producto_id', 'nombre', 'categoria', 'precio_costo', 'precio_venta
    on='producto_id'
)

# Calcular valores
df_transacciones['valor'] = np.where(
    df_transacciones['tipo'] == 'venta',
    df_transacciones['cantidad'] * df_transacciones['precio_venta'],
    df_transacciones['cantidad'] * df_transacciones['precio_costo']
)

print("SISTEMA DE GESTIÓN DE INVENTARIOS")
print("=" * 70)

# 1. Análisis de ventas por categoría
print("\n1. VENTAS POR CATEGORÍA")
print("-" * 70)

ventas_categoria = df_transacciones[df_transacciones['tipo'] == 'venta'].groupby('cat
    'valor': 'sum',
    'cantidad': 'sum',
    'producto_id': 'count'
}).round(2)

ventas_categoria.columns = ['valor_total', 'unidades_vendidas', 'num_transacciones']
ventas_categoria = ventas_categoria.sort_values('valor_total', ascending=False)
print(ventas_categoria)

# 2. Top 10 productos más vendidos
print("\n\n2. TOP 10 PRODUCTOS MÁS VENDIDOS")
print("-" * 70)

top_productos = df_transacciones[df_transacciones['tipo'] == 'venta'].groupby(
    ['producto_id', 'nombre']
).agg({
    'cantidad': 'sum',
    'valor': 'sum'
}).round(2)

top_productos.columns = ['unidades_vendidas', 'ingresos_totales']
top_productos = top_productos.sort_values('ingresos_totales', ascending=False).head(1
print(top_productos)

# 3. Productos bajo stock mínimo
print("\n\n3. ALERTA: PRODUCTOS BAJO STOCK MÍNIMO")
```

```
print("-" * 70)

productos_bajo_stock = df_productos[
    df_productos['stock_actual'] < df_productos['stock_minimo']
].copy()

productos_bajo_stock['faltante'] = (
    productos_bajo_stock['stock_minimo'] - productos_bajo_stock['stock_actual']
)

productos_bajo_stock = productos_bajo_stock[[
    'producto_id', 'nombre', 'categoria', 'stock_actual', 'stock_minimo', 'faltante'
]].sort_values('faltante', ascending=False)

print(productos_bajo_stock)

# 4. Análisis de rentabilidad
print("\n\n4. ANÁLISIS DE RENTABILIDAD POR PRODUCTO")
print("-" * 70)

ventas_por_producto = df_transacciones[df_transacciones['tipo'] == 'venta'].groupby(
    'producto_id'
).agg({
    'cantidad': 'sum',
    'precio_costo': 'first',
    'precio_venta': 'first'
})

ventas_por_producto['costo_total'] = (
    ventas_por_producto['cantidad'] * ventas_por_producto['precio_costo']
)
ventas_por_producto['ingresos'] = (
    ventas_por_producto['cantidad'] * ventas_por_producto['precio_venta']
)
ventas_por_producto['utilidad'] = (
    ventas_por_producto['ingresos'] - ventas_por_producto['costo_total']
)
ventas_por_producto['margen_pct'] = (
    ventas_por_producto['utilidad'] / ventas_por_producto['ingresos'] * 100
)

rentabilidad = ventas_por_producto[['cantidad', 'utilidad', 'margen_pct']].sort_value
    'utilidad', ascending=False
).head(10).round(2)

print(rentabilidad)
```

```python
# 5. Evolución mensual de ventas
print("\n\n5. EVOLUCIÓN MENSUAL DE VENTAS")
print("-" * 70)

df_transacciones['año_mes'] = df_transacciones['fecha'].dt.to_period('M')

ventas_mensuales = df_transacciones[df_transacciones['tipo'] == 'venta'].groupby('año
    'valor': 'sum',
    'cantidad': 'sum'
}).round(2)

ventas_mensuales.columns = ['ventas_soles', 'unidades']
ventas_mensuales['variacion_pct'] = ventas_mensuales['ventas_soles'].pct_change() * 1

print(ventas_mensuales.round(2))

# Guardar reportes
df_transacciones.to_csv('transacciones_inventario.csv', index=False)
productos_bajo_stock.to_csv('alerta_stock_bajo.csv', index=False)
print("\n\nReportes guardados:")
print("  - transacciones_inventario.csv")
print("  - alerta_stock_bajo.csv")
```

```
SISTEMA DE GESTIÓN DE INVENTARIOS
======================================================================

1. VENTAS POR CATEGORÍA
----------------------------------------------------------------------
             valor_total  unidades_vendidas  num_transacciones
categoria
Hogar          612605.58               1709                172
Alimentos      533735.35               2287                227
Electrónica    351288.52                855                 81
Ropa            45313.94                197                 20


2. TOP 10 PRODUCTOS MÁS VENDIDOS
----------------------------------------------------------------------
                        unidades_vendidas  ingresos_totales
producto_id nombre
PROD011     Producto 11               289         177044.29
PROD002     Producto 2                233         176332.07
PROD019     Producto 19               295         137434.60
PROD015     Producto 15               398         130540.02
PROD008     Producto 8                341         129331.07
PROD003     Producto 3                195         104009.10
PROD009     Producto 9                222          81141.00
PROD013     Producto 13               298          81118.58
```

```
PROD016     Producto 16                150          72969.00
PROD017     Producto 17                357          68087.04


3. ALERTA: PRODUCTOS BAJO STOCK MÍNIMO
----------------------------------------------------------------------
   producto_id       nombre    categoria  stock_actual  stock_minimo  faltante
15     PROD016  Producto 16  Electrónica            23            42        19
5      PROD006   Producto 6        Hogar            13            18         5
8      PROD009   Producto 9    Alimentos            14            17         3


4. ANÁLISIS DE RENTABILIDAD POR PRODUCTO
----------------------------------------------------------------------
             cantidad  utilidad  margen_pct
producto_id
PROD011           289  87509.20       49.43
PROD002           233  63266.49       35.88
PROD019           295  48852.00       35.55
PROD008           341  38239.74       29.57
PROD015           398  37618.96       28.82
PROD013           298  35479.88       43.74
PROD009           222  31934.70       39.36
PROD017           357  29588.16       43.46
PROD003           195  22518.60       21.65
PROD005           274  19958.16       42.37


5. EVOLUCIÓN MENSUAL DE VENTAS
----------------------------------------------------------------------
         ventas_soles  unidades  variacion_pct
año_mes
2023-01      90593.11       312            NaN
2023-02     121257.05       414          33.85
2023-03      90345.39       240         -25.49
2023-04     157379.32       510          74.20
2023-05     117624.03       482         -25.26
2023-06     161507.28       484          37.31
2023-07     129139.11       422         -20.04
2023-08     192384.53       577          48.97
2023-09     125496.86       401         -34.77
2023-10     134829.29       405           7.44
2023-11     104762.37       389         -22.30
2023-12     117625.05       412          12.28


Reportes guardados:
  - transacciones_inventario.csv
```

- `alerta_stock_bajo.csv`

13 Conclusión

En esta guía hemos explorado DataFrames con Pandas:
Creación de DataFrames
Desde diccionarios, listas, Series y con índices personalizados.
Importación y exportación
Lectura y escritura de CSV, Excel, JSON y otras fuentes de datos.
Selección y filtrado
Uso de loc, iloc, condiciones booleanas y query para filtrar datos.
Modificación de datos
Crear, modificar y eliminar columnas. Renombrar columnas eficientemente.
Operaciones
Aplicar funciones, agregaciones y transformaciones a los datos.
Merge y Join
Combinar DataFrames con diferentes tipos de joins (inner, left, right, outer).
Agrupación
GroupBy para análisis agregado y cálculo de estadísticas por grupos.

14 Próximos pasos

En la siguiente guía (Guía 8: Predicción y Métricas de Performance) exploraremos:

- El problema de predicción
- Matriz de confusión
- Métricas: Accuracy, Precision, Recall
- Curva ROC y AUC
- Evaluación de modelos de clasificación

Estas herramientas son fundamentales para evaluar modelos de machine learning aplicados a problemas económicos.

15 Recursos adicionales

Para profundizar en Pandas:

- Pandas Documentation: pandas.pydata.org/docs
- Pandas Cheat Sheet: pandas.pydata.org/Pandas_Cheat_Sheet.pdf
- Python for Data Analysis (Wes McKinney)
- Ejercicios con datos del INEI, BCRP y Banco Mundial

16 Publicaciones Similares

Si te interesó este artículo, te recomendamos que explores otros blogs y recursos relacionados que pueden ampliar tus conocimientos. Aquí te dejo algunas sugerencias:

1. Instalacion De Anaconda
2. Configurar Entorno Virtual Python Anaconda
3. 01 Introducion A La Programacion Con Python
4. 02 Variables Expresiones Y Statements Con Python

5. 03 Objetos De Python
6. 04 Ejecucion Condicional Con Python
7. 05 Iteraciones Con Python
8. 06 Funciones Con Python
9. 07 Dataframes Con Python
10. 08 Prediccion Y Metrica De Performance Con Python
11. 09 Metodos De Machine Learning Para Clasificacion Con Python
12. 10 Metodos De Machine Learning Para Regresion Con Python
13. Visualizacion De Datos Con Python

Esperamos que encuentres estas publicaciones igualmente interesantes y útiles. ¡Disfruta de la lectura!